

1. **Single-page apps and URLs:** Multi-page sites and single-page apps · The History API: changing the URL without a request · Refresh and deep links: every path must return index.html · Hash URLs and path URLs
2. **Routes and matching:** Setting up a router (React Router) · Routes: mapping a URL to a component · How the router picks a route: ranking, not order · URL parameters: useParams (always strings) · Query strings: useSearchParams · The catch-all route: a 404 page
3. **Nested routes and layouts:** Nested routes and layouts: <Outlet /> · Index routes: the default child
4. **Navigating:** <Link> instead of <a href> · Navigating from code: useNavigate · Push or replace: what the Back button will do
5. **Protected routes and data:** Protected routes: a guard component · Back to where you were after log-in · Loading data for a route

Single-page apps and URLs

Multi-page sites and single-page apps

In a **multi-page site** every link asks the server for a new HTML page. In a **single-page app (SPA)** the server sends one HTML page and the JavaScript bundle once; after that, the app itself swaps screens and changes the URL, asking the server only for data.

- SPA: **one** HTML page, many screens, each with its own URL.
- After the first load, an SPA asks the server for **data**, not pages.
- The **client-side router** keeps the URL and the screen in step.

What happens when the user clicks "Task 7"

	Multi-page site	Single-page app
Request	GET /tasks/7 → a whole HTML page	none for the page; GET /api/tasks/7 → JSON
Who builds the screen	the server	the app, in the browser
JavaScript state (variables, forms)	lost	kept
The URL changes because...	the browser loaded a new document	the router called history.pushState
Back button	the browser loads the previous page	the router draws the previous screen
First visit	fast: one ready-made page	slower: the bundle must download and run first

Example. Open the Network tab of the browser's developer tools (see [DevTools: the Network tab](#)) on any React app and click around: the "Doc" filter shows one request, the first one; the "Fetch/XHR" filter shows the JSON calls each screen makes. Do the same on a classic multi-page site: every click adds a new "Doc" request.

Common mistake. Thinking that a single-page app has a single URL. A good SPA has as many URLs as screens (/tasks, /tasks/7, /tasks/7/edit); "single page" refers to the one HTML document the server sends, not to the address bar.

The History API: changing the URL without a request

The browser's **History API** lets JavaScript change the URL without loading a page: history.pushState(state, '', '/tasks/7') **adds an entry to the history**, replaceState **overwrites the current one**, and the popstate **event fires when the user presses Back or Forward**. Every client-side router is built on these three.

```
// A hand-made router in plain JavaScript: what a library like React Router does underneath
const routes = {
  '/': () => '<h1>Home</h1>',
  '/tasks': () => '<h1>All tasks</h1>',
};

function render() {
  const page = routes[location.pathname] || (() => '<h1>Not found</h1>');
  document.querySelector('#app').innerHTML = page();
}

// 1. Links: change the URL without a request, then draw
document.addEventListener('click', (e) => {
  const a = e.target.closest('a[data-link]');
```

```

    if (!a) return;
    e.preventDefault();                // stop the browser loading a page
    history.pushState(null, '', a.getAttribute('href'));
    render();                          // pushState does not fire popstate
  });

// 2. Back and Forward: the browser fires popstate, we draw the URL it restored
window.addEventListener('popstate', render);

render();                             // 3. the first load

```

Example. With the code above, on / the user clicks : the listener cancels the navigation, pushState makes the history [/ , /tasks] with the pointer on /tasks, and render() draws "All tasks". Back moves the pointer to / and fires popstate, so render() draws "Home". No request reached the server in either step.

Common mistake. Expecting pushState to show the new page by itself, or to fire popstate. It only changes the address bar and the history; if your code does not render afterwards, the URL says /tasks while the screen still shows the home page.

Refresh and deep links: every path must return index.html

A client-side route such as /tasks/7 exists only inside the app. When the user **refreshes** or opens a shared link, the browser asks the **server** for /tasks/7; unless the server answers every app path with index.html (the **SPA fallback**), the user gets a 404 instead of the app.

```

// Express serving the built React app (the dist/ folder from npm run build)
const path = require('path');
const dist = path.join(__dirname, '..', 'client', 'dist');

app.use('/api', apiRouter);           // 1. the API first
app.use(express.static(dist));        // 2. real files: /assets/index-3f2a.js ...

// 3. the SPA fallback: any other GET that is not /api/... gets index.html
app.get(/^\/(?!api\/).*/, (req, res) => {
  res.sendFile(path.join(dist, 'index.html'));
});

app.use((req, res) => res.status(404).json({ error: 'Not found' })); // 4. unknown /api/... routes

```

Example. In the router simulator, switch off "The server answers every path with index.html", click a link to /tasks (it works: no request is made), then press **Refresh**: the page becomes the server's 404 Not Found · Cannot GET /tasks, and the history shows a new page load. Switch the fallback on and refresh again: the app starts and draws /tasks.

Common mistake. Making the fallback catch everything, API included: GET /api/tsks (a typo) then answers 200 with an HTML page, and the front end fails with "Unexpected token '<' in JSON". Exclude /api/ from the fallback so unknown API paths keep their JSON 404 (see [The 404 catch-all](#)).

Hash URLs and path URLs

A router can keep the screen in the **path** (/tasks/7, needs the server fallback) or in the **hash** (/#/tasks/7). The hash, everything after #, is never sent to the server, so hash URLs work on any static host, at the price of uglier URLs.

The same screen, two kinds of URL

	Path: /tasks/7	Hash: /#/tasks/7
What the server receives	GET /tasks/7	GET / (the hash is not sent)
Server configuration	SPA fallback needed	none
Browser API underneath	pushState + popstate	location.hash + hashchange
React Router	BrowserRouter / createBrowserRouter	HashRouter / createHashRouter
In-page anchors (#section)	work as usual	clash with the route

```
// A hash router in plain JavaScript: no server configuration needed
function render() {
  const path = location.hash.slice(1) || '/';    // '#/tasks/7' → '/tasks/7'
  document.querySelector('#app').textContent = 'Screen for ' + path;
}
window.addEventListener('hashchange', render);  // links: <a href="#/tasks/7">
render();
```

Example. Deploy the same app twice to a static host without rewrites. With `BrowserRouter`, `https://me.github.io/app/tasks/7` gives a 404 on refresh. With `HashRouter`, `https://me.github.io/app/#/tasks/7` always works: the server only ever sees `/app/`.

Common mistake. Switching to `HashRouter` while keeping `href="/tasks"` in plain `<a>` tags. Those still point at a path, so they trigger a full page load to a URL the host does not have. Use the router's `<Link to="/tasks">`, which writes the right URL for the router you chose.

Routes and matching

Setting up a router (React Router)

In React, a router is a component at the top of the app that listens to the URL and gives every component below it access to the current location. With **React Router**: `npm install react-router`, wrap the app in `<BrowserRouter>`, and describe the screens with `<Routes>` and `<Route>`.

```
// main.jsx: one router, at the top
import { createRoot } from 'react-dom/client';
import { BrowserRouter } from 'react-router';    // version 6: 'react-router-dom'
import App from './App.jsx';

createRoot(document.getElementById('root')).render(
  <BrowserRouter>
    <App />
  </BrowserRouter>
);

// App.jsx: which component renders for which URL
import { Routes, Route } from 'react-router';
import Home from './pages/Home.jsx';
import TaskList from './pages/TaskList.jsx';
import TaskDetail from './pages/TaskDetail.jsx';

export default function App() {
  return (
    <Routes>
      <Route path="/" element={<Home />} />
      <Route path="/tasks" element={<TaskList />} />
      <Route path="/tasks/:id" element={<TaskDetail />} />
    </Routes>
  );
}

// The same routes as objects (data mode)
// const router = createBrowserRouter([
//   { path: '/', element: <Home /> },
//   { path: '/tasks', element: <TaskList /> },
//   { path: '/tasks/:id', element: <TaskDetail /> },
// ]);
// createRoot(root).render(<RouterProvider router={router} />);
```

Example. A typical layout of files: `src/pages/` holds one component per screen (`TaskList.jsx`, `TaskDetail.jsx`, `Login.jsx`), `src/components/` the pieces they share (`TaskItem`, `NavBar`), and `App.jsx` the route table that connects URLs to pages. Reading `App.jsx` alone tells you every screen the app has.

Common mistake. Calling `useNavigate()` in a component that is rendered **outside** the router, for example an auth provider placed above `<BrowserRouter>` in `main.jsx`. React Router throws "useNavigate() may be used only in the context of a `<Router>` component". Put the router at the very top, and any provider that navigates inside it.

Routes: mapping a URL to a component

A route pairs a **URL pattern** (`path="/tasks/:id"`) with what to render for it (`element={<TaskDetail />}`). `<Routes>` looks at the current URL, chooses the best matching route and renders its element; when the URL changes, it chooses again.

```
import { useState } from 'react';

// A hand-made router, to see the idea: the URL lives in state,
// and a table of routes says what to render for it.
// (React Router reads the real URL; these boxes cannot change it.)
const routes = [
  { path: '/', element: <h2>Home</h2> },
  { path: '/tasks', element: <h2>All tasks</h2> },
  { path: '/about', element: <h2>About this app</h2> },
];

function App() {
  const [path, setPath] = useState('/');
  const route = routes.find((r) => r.path === path.toLowerCase().replace(/(.)\$/ , '$1'));
  return (
    <>
      <nav style={{ display: 'flex', flexWrap: 'wrap', gap: 6 }}>
        {[ '/', '/tasks', '/about', '/Tasks', '/nope' ].map((p) => (
          <button key={p} onClick={() => setPath(p)} aria-pressed={p === path}>{p}</button>
        ))}
      </nav>
      <p>URL: <code>{path}</code></p>
      {route ? route.element : <p>(nothing: no route matches)</p>}
    </>
  );
}
```

Example. In the box, click `/Tasks/`: it still shows "All tasks", because the toy matcher, like React Router, ignores case and a trailing slash. Click `/nope`: no route matches and nothing renders, which is what React Router does too (plus a console warning `No routes matched location "/nope"`).

Common mistake. Writing routes in the React Router 5 style found in old tutorials: `<Route exact path="/" component={Home} />` inside a `<Switch>`. In current versions `Switch` does not exist (it is `Routes`), `exact` is gone (every path is exact unless it ends in `/*`) and `component` is ignored, so the route renders nothing. Write `<Route path="/" element={<Home />} />`.

How the router picks a route: ranking, not order

When several routes match a URL, React Router does not take the first one written: it **ranks** them, and the most specific path wins. A literal segment beats a `:param`, which beats `*`. So `/tasks/new` and `/tasks/:id` can be written in any order.

Scores for some paths (React Router 6 and 7)

Path	How the score is made	Score
<code>/tasks/:id/edit</code>	4 segments + 1 + 10 + 3 + 10	28
<code>/tasks/new</code>	3 segments + 1 + 10 + 10	24
<code>/tasks/:id</code>	3 segments + 1 + 10 + 3	17
<code>/tasks (index child)</code>	3 segments + 2 (index) + 1 + 10 + 1	17
<code>/tasks</code>	2 segments + 1 + 10	13
<code>/</code>	2 segments + 1 + 1	4
<code>*</code>	2 segments – 2 (splat) + 1	1

Example. In the simulator above (example "A task app"), type `/tasks/new` and click the Link: the ranking table at the bottom shows `/tasks/:id/edit` (28) tried and rejected, then `/tasks/new` (24) matching. `/tasks/:id` (17) is never tried. Challenge 2 asks you to predict this, and challenge 6 shows the one case where order decides.

Common mistake. Carrying the Express habit over and worrying that `:id` "swallows" `new`, then renaming the route to `/new-task` or adding checks such as `if (id === 'new')` in `TaskDetail`. React Router already picks `/tasks/new`; the workaround only adds confusion.

URL parameters: useParams (always strings)

A :name segment in a route path captures that part of the URL. Inside the route's components, `useParams()` returns them as an object: for `path="/tasks/:id"` and the URL `/tasks/7`, `useParams()` is `{ id: '7' }`, a string.

```
// App.jsx
<Route path="/tasks/:id" element={TaskDetail} /> />

// TaskDetail.jsx
import { useParams } from 'react-router';

export default function TaskDetail() {
  const { id } = useParams();           // '7' for /tasks/7: a string
  const taskId = Number(id);
  if (!Number.isInteger(taskId) || taskId < 1) return <p>There is no task "{id}".</p>;
  return <h1>Task {taskId}</h1>;
}

import { useState } from 'react';

// A hand-made matcher for one pattern, to see what useParams() returns.
function matchPath(pattern, path) {
  const want = pattern.split('/');
  const got = path.split('/');
  if (want.length !== got.length) return null;
  const params = {};
  for (let i = 0; i < want.length; i++) {
    if (want[i].startsWith(':')) {
      if (!got[i]) return null;           // a param needs a non-empty segment
      try { params[want[i].slice(1)] = decodeURIComponent(got[i]); } catch { return null; }
    } else if (want[i] !== got[i]) return null;
  }
  return params;
}

function App() {
  const [path, setPath] = useState('/tasks/7');
  const params = matchPath('/tasks/:id', path);
  return (
    <>
      <label>URL <input value={path} onChange={(e) => setPath(e.target.value)} /></label>
      <p>Pattern: <code>/tasks/:id</code></p>
      {params ? (
        <ul>
          <li>params: <code>{JSON.stringify(params)}</code></li>
          <li>typeof params.id: <code>{typeof params.id}</code></li>
          <li>params.id === 7: <code>{String(params.id === 7)}</code></li>
          <li>Number(params.id) === 7: <code>{String(Number(params.id) === 7)}</code></li>
        </ul>
      ) : <p>No match: the URL has a different shape.</p>}
    </>
  );
}
```

Example. In the box, the URL `/tasks/7` gives `{"id": "7"}`: `typeof` is string, `params.id === 7` is false and `Number(params.id) === 7` is true. Type `/tasks/buy%20milk`: the param is decoded to `buy milk`. Type `/tasks/7/edit` or `/tasks/`: no match, the shape is different.

Common mistake. Comparing the param with a number from the data: `tasks.find((t) => t.id === id)` finds nothing, because `t.id` is the number 7 and `id` is the string '7', and the page shows "not found" for a task that exists. Convert once, `const taskId = Number(id)`, and compare numbers.

Query strings: `useSearchParams`

The query string (`?done=false&page=2`) holds **optional settings of a screen: filters, search text, sort, page**. React Router does not use it to choose a route; the page reads and changes it with `const [searchParams, setSearchParams] = useSearchParams()`.

```
import { useState } from 'react';

const TASKS = [
  { id: 1, title: 'Buy milk', done: true },
  { id: 2, title: 'Call Ana', done: false },
  { id: 3, title: 'Write the report', done: false },
];

function App() {
  // A hand-made URL in state; useSearchParams() would read the real one.
  const [url, setUrl] = useState('/tasks?done=false');
  const [pathname, query = ''] = url.split('?');
  const searchParams = new URLSearchParams(query);
  const done = searchParams.get('done'); // 'true', 'false' or null
  const q = searchParams.get('q') ?? '';

  function update(key, value) { // what setSearchParams does
    const next = new URLSearchParams(searchParams);
    if (value) next.set(key, value); else next.delete(key);
    const s = next.toString();
    setUrl(pathname + (s ? '?' + s : ''));
  }

  const shown = TASKS.filter((t) => (done === null || String(t.done) === done)
    && t.title.toLowerCase().includes(q.toLowerCase()));
  return (
    <>
      <p>URL: <code>{url}</code></p>
      <label>Search <input value={q} onChange={(e) => update('q', e.target.value)} /></label>{' '}
      <select value={done ?? ''} onChange={(e) => update('done', e.target.value)} aria-label="Status">
        <option value="">All</option><option value="false">To do</option><option value="true">Done</option>
      </select>
      <ul>{shown.map((t) => <li key={t.id}>{t.title}</li>)}</ul>
    </>
  );
}
```

Example. In the box, choose "All": the done key disappears from the URL and all three tasks show. Type "rep" in Search: the URL becomes `/tasks?q=rep` and the list follows. Every filter is in the URL, so a reload (or a shared link) would show exactly this list.

Common mistake. Writing the query string into the route path, `path="/tasks?done=:done"`. Route paths never include the query string (React Router matches only the path), so this route never matches. Keep `path="/tasks"` and read the filter with `useSearchParams`.

The catch-all route: a 404 page

A route with `path="*" matches any URL, but it has the lowest score, so it only renders when no other route matches. Use it for a "Page not found" screen instead of a blank page.`

```
<Routes>
  <Route path="/" element={<AppLayout />}>
    <Route index element={<Home />} />
    <Route path="tasks" element={<TaskList />} />
    <Route path="tasks/:id" element={<TaskDetail />} />
  </Route>
  <Route path="*" element={<PageNotFound />} />
</Routes>
```

```

    <Route path="*" element={<NotFound />} />    {/* last resort, inside the layout */}
  </Route>
</Routes>

function NotFound() {
  const location = useLocation();
  return (
    <section>
      <h1>Page not found</h1>
      <p>Nothing lives at <code>{location.pathname}</code>.</p>
      <Link to="/tasks">Back to your tasks</Link>
    </section>
  );
}

```

Example. With the routes above, `/tsks` and `/tasks/7/oops` both render `AppLayout` with `NotFound` in its outlet: neither matches a real route (`/tasks/:id` takes exactly one segment after `tasks`), so only `*` is left.

Common mistake. Redirecting unknown URLs to the home page with `<Route path="*" element={<Navigate to="/" />} />`. The user who mistyped never learns that the page does not exist, and a broken link in your own app becomes impossible to notice. Show a real not-found page.

Nested routes and layouts

Nested routes and layouts: <Outlet />

Routes can contain routes. The parent route's element is a `layout` (header, navigation, sidebar) and renders `<Outlet />` where the matching child goes. The layout stays mounted while the user moves between its children; only the outlet's content changes.

```

// App.jsx: the route tree mirrors the screen tree
<Routes>
  <Route path="/" element={<AppLayout />}>
    <Route index element={<Home />} />
    <Route path="tasks" element={<TasksLayout />}>
      <Route index element={<TaskList />} />
      <Route path=":id" element={<TaskDetail />} />
    </Route>
  </Route>
</Routes>

// AppLayout.jsx: drawn on every page
import { Outlet, NavLink } from 'react-router';

export default function AppLayout() {
  return (
    <>
      <header>
        <nav>
          <NavLink to="/" end>Home</NavLink>
          <NavLink to="/tasks">Tasks</NavLink>
        </nav>
      </header>
      <main>
        <Outlet />    {/* the matching child route renders here */}
      </main>
    </>
  );
}

```

```

import { useState } from 'react';

// Hand-made nesting: the layout receives the child page as "outlet"
// (in React Router it would write <Outlet /> instead).
function AppLayout({ outlet, go }) {

```

```
const [note, setNote] = useState('');
return (
  <div style={{ border: '2px solid teal', padding: 8 }}>
    <header>
      <strong>Tasks app</strong>{' '}
      <button onClick={() => go('/')}>Home</button>
      <button onClick={() => go('/tasks')}>Tasks</button>
      <button onClick={() => go('/tasks/2')}>Task 2</button>
      <p><input placeholder="Type here, then navigate" value={note}
        onChange={(e) => setNote(e.target.value)} /></p>
    </header>
    <main style={{ border: '2px dashed orange', padding: 8 }}>{outlet}</main>
  </div>
);
}

function App() {
  const [path, setPath] = useState('/tasks');
  let page = <p>Not found</p>;
  if (path === '/') page = <h2>Home</h2>;
  else if (path === '/tasks') page = <h2>All tasks</h2>;
  else if (path.startsWith('/tasks/')) page = <h2>Task {path.split('/')[2]}</h2>;
  return <AppLayout outlet={page} go={setPath} />;
}
```

Example. In the box, type something in the header's input, then click Home, Tasks and Task 2: only the dashed area (the outlet) changes and your text stays, because AppLayout is never unmounted. With every page rendering its own copy of the header, the input would be recreated, and emptied, on every click.

Common mistake. Nesting the routes but forgetting `<Outlet />` in the parent's element. The URL changes, the parent renders, and the child silently does not appear: there is no error, the router simply has nowhere to put it. If a child route "does not work", check that every element on the way down renders an outlet.

Index routes: the default child

An index route, `<Route index element={<TaskList />} />`, **renders in its parent's outlet when the URL is exactly the parent's path. It has no path of its own: it fills the outlet at /tasks, while /tasks/7 shows another child.**

```
<Route path="tasks" element={<TasksLayout />}>
  <Route index element={<TaskList />} />    {/* /tasks */}
  <Route path="new" element={<NewTask />} />  {/* /tasks/new */}
  <Route path=":id" element={<TaskDetail />} /> {/* /tasks/7 */}
</Route>
```

Example. With the routes above, /tasks renders TasksLayout with TaskList in the outlet. Remove the index route and /tasks still matches (the parent has a path), but the outlet is empty: the layout's title and buttons with a blank area below.

Common mistake. Giving the index route the parent's path again, `<Route path="tasks" element={<TaskList />} />` inside `<Route path="tasks">`. Child paths are relative, so this child lives at /tasks/tasks, and /tasks still has an empty outlet. Use index.

Navigating

<Link> instead of <a href>

Inside the app, navigate with `<Link to="/tasks">`. It renders a normal `<a href>` (so it is accessible and can be opened in a new tab), but a click is handled by the router: no page load, no request, the app's state is kept. A plain `<a href="/tasks">` reloads the whole app.

```
import { Link, NavLink } from 'react-router';

function NavBar() {
  return (
```

```

    <nav aria-label="Main">
      <NavLink to="/" end>Home</NavLink>
      <NavLink to="/tasks">Tasks</NavLink>
      <NavLink to="/tasks/new" className={({ isActive }) => (isActive ? 'tab current' : 'tab')}>
        New task
      </NavLink>
    </nav>
  );
}

// In a list: a link per item, built from the data
<li key={task.id}><Link to={"/tasks/" + task.id}>{task.title}</Link></li>

/* CSS: highlight the current page */
/* nav a.active, nav a[aria-current="page"] { font-weight: 700; border-bottom: 3px solid; } */

```

```

import { useState } from 'react';

// Simulated: a full page load is a new start of the app,
// imitated here by giving the app a new key (React remounts it from zero).
function TheApp({ path, go, fullLoad }) {
  const [clicks, setClicks] = useState(0); // state kept in memory
  return (
    <>
      <p>State in memory: <strong>{clicks}</strong>{' '}</p>
      <button onClick={() => setClicks(clicks + 1)}>+1</button></p>
      <p>
        <button onClick={() => go('/tasks')}>{<Link to="/tasks">}</button>{' '}
        <button onClick={() => fullLoad('/tasks')}>{<a href="/tasks">}</button>{' '}
        <button onClick={() => go('/')}>{<Link to="/">}</button>
      </p>
      <p>Page: <code>{path}</code></p>
    </>
  );
}

function App() {
  const [path, setPath] = useState('/');
  const [loads, setLoads] = useState(1);
  function fullLoad(to) {
    setPath(to);
    setLoads(loads + 1); // the browser starts the app again
  }
  return (
    <>
      <p>Page loads (requests for index.html): {loads}</p>
      <TheApp key={loads} path={path} go={setPath} fullLoad={fullLoad} />
    </>
  );
}

```

Example. In the box, press +1 a few times, then use the two "Link" buttons: the page changes and the counter stays. Press the <a href> button: "Page loads" goes up and the counter is back to 0, because the whole app started again, which is what a real <a href> inside an SPA does to all its state.

Common mistake. Using <button onClick={() => navigate('/tasks/' + id)}> for plain navigation in a list. It works with a mouse, but the user cannot open the task in a new tab, cannot see where it leads, and assistive technology announces a button, not a link. If clicking it goes somewhere, it is a <Link>.

Navigating from code: useNavigate

`const navigate = useNavigate()` gives a function that changes the URL from code: `navigate('/tasks/12')` after a form is saved, `navigate('/login')` after logging out, `navigate(-1)` for "back". Use it when the navigation is the result of an action, not a place the user clicks.

```
import { useState } from 'react';

// Hand-made: the history is an array of URLs; navigate() adds one (push)
// or overwrites the last one (replace). This toy Back forgets the forward page.
function App() {
  const [stack, setStack] = useState(['tasks', 'tasks/new']);
  const [tasks, setTasks] = useState([{ id: 1, title: 'Buy milk' }]);
  const [title, setTitle] = useState('');
  const path = stack[stack.length - 1];
  const navigate = (to, { replace = false } = {}) =>
    setStack((s) => (replace ? [...s.slice(0, -1), to] : [...s, to]));

  function handleSubmit(e) {
    e.preventDefault();
    const task = { id: tasks.length + 1, title };
    setTasks([...tasks, task]);
    setTitle('');
    navigate('/tasks/' + task.id);          // try { replace: true }
  }

  let page = <ul>{tasks.map((t) => <li key={t.id}>{t.title}</li>)}</ul>;
  if (path === '/tasks/new') page = (
    <form onSubmit={handleSubmit}>
      <label>Title <input value={title} onChange={(e) => setTitle(e.target.value)} required /></label>
      <button>Save</button>
    </form>
  );
  else if (path !== '/tasks') page = <h2>{tasks.find((t) => '/tasks/' + t.id === path)?.title}</h2>;
  return (
    <>
      <p><button onClick={() => setStack((s) => s.slice(0, -1))} disabled={stack.length < 2}>Back</button>{' '}
        <code>{path}</code></p>
      {page}
      <p>History: {stack.join(' → ')}</p>
    </>
  );
}
```

Example. In the box, save a task called "Call Ana": the app shows it at `/tasks/2` and the history is `/tasks → /tasks/new → /tasks/2`. Press Back: you land on the empty form again, which is rarely what users want. Change the call to `navigate('/tasks/' + task.id, { replace: true })` and repeat: the form's entry is overwritten, and Back goes straight to the list.

Common mistake. Navigating before the work is done: `api.createTask(values); navigate('/tasks');` without `await`. The list page mounts and fetches while the POST is still on its way, so the new task is missing, and if the POST fails the user never sees the error. `await` the request, check it succeeded, then `navigate`.

Push or replace: what the Back button will do

Every navigation either **pushes** a new history entry (Back returns to the current page) or **replaces** the current one (Back skips it). Use `replace` for pages the user should not come back to: redirects, the log-in page after logging in, a form after it was saved.

History after logging in: the user started on `/tasks` and clicked Settings

Guard	After log-in	History	Back from <code>/settings</code> goes to
push	push	<code>/tasks → /settings → /login → /settings</code>	<code>/login</code> (the form, already logged in)
replace	push	<code>/tasks → /login → /settings</code>	<code>/login</code>
push	replace	<code>/tasks → /settings → /settings</code>	<code>/settings</code> (nothing seems to happen)

Guard	After log-in	History	Back from /settings goes to
replace	replace	/tasks → /settings	/tasks

Example. In the router simulator, open challenge 8: both switches start off. Click Settings while logged out, log in, then press Back and watch the history stack: you land on /login. Switch on the guard's replace and the log-in replace, run the story again, and Back returns to /tasks.

Common mistake. Using replace everywhere "to keep the history clean". Then Back skips pages the user did want: from a task's detail, Back leaves the app instead of returning to the list. Push is the default for a reason; replace only the entries that should not be revisited.

Protected routes and data

Protected routes: a guard component

A protected route renders its page only for a logged-in user. It is a small wrapper component, often called `RequireAuth`: it reads the auth state and renders the page (or `<Outlet />`), or `<Navigate to="/login" replace />` when nobody is logged in.

```
// RequireAuth.jsx
import { Navigate, Outlet, useLocation } from 'react-router';
import { useAuth } from './AuthContext.jsx';

export default function RequireAuth({ children }) {
  const { user, checking } = useAuth();
  const location = useLocation();
  if (checking) return <p>Loading...</p>; // still reading the saved token
  if (!user) {
    return <Navigate to="/login" replace state={{ from: location }} />;
  }
  return children ?? <Outlet />; // wraps one page, or a group of child routes
}

// App.jsx
<Route path="/" element={<AppLayout />}>
  <Route index element={<Home />} />
  <Route path="login" element={<Login />} /> // /* public, never guarded */
  <Route element={<RequireAuth />}> // /* every route inside is private */
    <Route path="tasks/new" element={<NewTask />} />
    <Route path="tasks/:id/edit" element={<EditTask />} />
    <Route path="settings" element={<Settings />} />
  </Route>
</Route>
```

```
import { useEffect, useState } from 'react';

// Hand-made pieces that imitate React Router: the location in state,
// a <Navigate> that redirects once it renders, and a guard.
function Navigate({ to, state, navigate }) {
  useEffect(() => { navigate(to, { state }); }, [to]);
  return null;
}

function RequireAuth({ user, location, navigate, children }) {
  if (!user) return <Navigate to="/login" state={{ from: location.path }} navigate={navigate} />;
  return children;
}

function App() {
  const [location, setLocation] = useState({ path: '/', state: null });
  const [user, setUser] = useState(null);
  const navigate = (path, opts = {}) => setLocation({ path, state: opts.state ?? null });

  let page = <h2>Home (public)</h2>;
  if (location.path === '/settings') page = (
```

```

    <RequireAuth user={user} location={location} navigate={navigate}>
      <h2>Settings of {user?.name}</h2>
    </RequireAuth>
  );
  if (location.path === '/login') page = (
    <button onClick={() => { setUser({ name: 'Ana' }); navigate(location.state?.from ?? '/'); }}>
      Log in as Ana
    </button>
  );
  return (
    <>
      <nav>
        <button onClick={() => navigate('/')}>Home</button>{' '}
        <button onClick={() => navigate('/settings')}>Settings</button>{' '}
        {user && <button onClick={() => setUser(null)}>Log out</button>}
      </nav>
      <p>URL <code>{location.path}</code> · state <code>{JSON.stringify(location.state)}</code> · user: {user ?
user.name : 'nobody'}</p>
      {page}
    </>
  );
}

```

Example. In the box, click Settings while logged out: the guard renders the redirect, the URL becomes `/login` and the state remembers `{ "from": "/settings" }`. Log in: you land on Settings. Click Log out while on Settings: the guard runs again and sends you back to `/login`.

Common mistake. Starting the auth state as "logged out" and restoring the saved token in an effect. On a refresh of `/settings`, the first render has no user yet, the guard redirects to `/login`, and only then does the effect find the token: logged-in users are thrown out on every reload. Read the token synchronously in the initial state (`useState(() => localStorage.getItem('token'))`), or keep a checking flag and render "Loading..." until it is known, as the code above does.

Back to where you were after log-in

When the guard redirects, it stores the page the user wanted in the navigation **state**: `<Navigate to="/login"`
 replace state={{ from: location }} />. After a successful log-in, the log-in page reads it with
`useLocation().state?.from` and navigates there, with `replace`.

```

// Login.jsx
import { useState } from 'react';
import { useLocation, useNavigate } from 'react-router';
import { useAuth } from './AuthContext.jsx';

export default function Login() {
  const { login } = useAuth();
  const navigate = useNavigate();
  const location = useLocation();
  const from = location.state?.from;
  const target = from ? from.pathname + from.search : '/tasks';
  const [error, setError] = useState(null);

  async function handleSubmit(e) {
    e.preventDefault();
    const form = new FormData(e.currentTarget);
    try {
      await login(form.get('email'), form.get('password'));
      navigate(target, { replace: true }); // back where they wanted to go
    } catch (err) {
      setError(err.message); // stay here and explain
    }
  }

  return (
    <form onSubmit={handleSubmit}>
      {from && <p>Log in to see {from.pathname}</p>}
      { /* email and password fields, the error, a submit button */ }
    </form>
  );
}

```

```

    </form>
  );
}

```

Example. A colleague sends a link to `/tasks/12/edit`. Logged out, the user opens it: the guard redirects to `/login` with `state.from = { pathname: '/tasks/12/edit', search: '' }`, and the form says "Log in to see `/tasks/12/edit`". After logging in, `navigate('/tasks/12/edit', { replace: true })` opens the edit page, and Back leaves for the page they came from, not the log-in form.

Common mistake. Putting the destination in the query string, `/login?next=/tasks/12`, and navigating to it without checking. Anyone can then craft a log-in link that sends users to `?next=https://evil.example` after logging in (an "open redirect"). Prefer history state, or accept only paths that start with a single `/`.

Loading data for a route

A page for one resource reads the param (`useParams()`) and fetches that resource in an effect with the param in its dependencies: `useEffect(..., [id])`. When the user moves from `/tasks/1` to `/tasks/2`, the same component stays mounted, so only the dependency makes it fetch again.

```

import { useEffect, useState } from 'react';

function TaskDetail({ id }) { // in React Router: const { id } = useParams();
  const [task, setTask] = useState(null);
  const [error, setError] = useState(null);
  useEffect(() => {
    let ignore = false;
    setTask(null);
    setError(null);
    fetch('/api/tasks/' + id)
      .then(async (res) => {
        const body = await res.json();
        if (!res.ok) throw new Error(res.status === 404 ? 'This task does not exist.' : body.error);
        return body;
      })
      .then((t) => { if (!ignore) setTask(t); })
      .catch((e) => { if (!ignore) setError(e.message); });
    return () => { ignore = true; }; // a newer id arrived: ignore this answer
  }, [id]); // a new id in the URL → fetch again
  if (error) return <p role="alert">Task {id}: {error}</p>;
  if (!task) return <p>Loading task {id}...</p>;
  return <h2>#{task.id} {task.title} {task.done ? '(done)' : ''}</h2>;
}

function App() {
  const [path, setPath] = useState('/tasks/1');
  const id = path.split('/')[2];
  return (
    <>
      <nav style={{ display: 'flex', flexWrap: 'wrap', gap: 6 }}>
        {[1, 2, 3, 999, 'abc'].map((n) => (
          <button key={n} onClick={() => setPath('/tasks/' + n)}>/tasks/{n}</button>
        ))}
      </nav>
      <p>URL: <code>{path}</code></p>
      <TaskDetail id={id} />
    </>
  );
}

```

Example. In the box, click through `/tasks/1`, `/tasks/2` and `/tasks/3`: each click shows "Loading..." then the task, because `id` changed and the effect ran again (the console lists one request per click). `/tasks/999` shows "This task does not exist." (the API answered 404) and `/tasks/abc` the API's validation error (400). Change `[id]` to `[]` and click around: the URL changes, the task does not.

Common mistake. Fetching all tasks on the detail page and picking one, `tasks.find((t) => t.id === id)`, or relying on the list page to pass the task in navigation state. The first downloads everything to show one item (and compares a number with the string param); the second breaks as soon as someone opens `/tasks/7` from a bookmark. Fetch the one resource the URL names.